

Assinatura e Verificação de Artefatos

Construção de uma Cadeia de Confiança para Build e Deploy
com Cosign e Sigstore

Disciplina:	Segurança de Software
Entrega:	Relatório Parcial – 28/06/2025
Grupo:	13
Integrantes:	Miguel Ángel Mantilla García - Santiago Maszong
Tópico:	Engenharia Segura e DevSecOps

Sumário

Construção de uma Cadeia de Confiança para Build e Deploy com Cosign e Sigstore	1
1 Introdução	3
2 Desenvolvimento	3
2.1 Fundamentos de Assinatura Digital	3
Algoritmos utilizados pelo Cosign:	5
2.2 O Problema da Cadeia de Suprimentos de Software	5
Níveis de maturidade SLSA:	5
2.3 Sigstore: Infraestrutura de Confiança Open-Source	6
2.4 Cosign: Assinatura de Imagens de Container	6
2.5 Rekor: Log de Transparência Imutável	7
2.6 Fulcio: Autoridade Certificadora sem Chaves de Longa Duração	7
3 Experimento Planejado	8
3.1 Descrição da Implementação	8
Componentes do experimento:	8
3.2 Pseudocódigo e Diagrama do Pipeline	9
Diagrama de componentes do experimento:	9
4 Referências Bibliográficas	10

1 Introdução

Em março de 2024, o pesquisador Andres Freund descobriu um backdoor cuidadosamente inserido na biblioteca de compressão **XZ Utils** (CVE-2024-3094). O agente malicioso, operando sob pseudônimo durante meses, ganhou a confiança dos mantenedores do projeto e comprometeu simultaneamente o código-fonte e o próprio processo de build: o repositório continha scripts que, durante a compilação, injetavam o backdoor nos binários gerados, de modo que o código aparentemente inofensivo produzia artefatos comprometidos [CISA, 2024]. Esse incidente expõe duas classes distintas de ameaças à cadeia de suprimentos de software: o comprometimento do processo de build (que a assinatura de artefatos, por si só, não teria evitado) e a distribuição de artefatos adulterados após o build (que é exatamente o que Cosign e Sigstore se propõem a detectar e impedir).

Este trabalho aborda a segunda classe de ameaça: a **assinatura e verificação criptográfica de artefatos de software**. O objetivo é implementar uma cadeia de confiança completa, desde o build até o deploy, utilizando as ferramentas **Cosign** e **Sigstore**. Essas ferramentas, desenvolvidas e mantidas pela Linux Foundation, representam o estado da arte em segurança de cadeia de suprimentos de software (*software supply chain security*) e são adotadas por projetos como Kubernetes, PyPI e npm [Sigstore, 2024]. É importante delimitar o escopo: se o próprio pipeline estiver comprometido, como ocorreu no XZ Utils, a assinatura será gerada sobre um artefato já malicioso e continuará válida. A proteção contra esse vetor requer abordagens complementares, como builds reproduzíveis e os níveis mais avançados do framework SLSA, discutidos na Seção 2.2.

A motivação para essa escolha é dupla. Do ponto de vista teórico, o tema conecta diretamente conceitos fundamentais de criptografia (assinaturas digitais, infraestrutura de chave pública, logs de transparência) com aplicações práticas modernas. Do ponto de vista prático, a implementação de assinatura de artefatos é uma necessidade crescente na indústria, impulsionada por regulamentações como o Executive Order on Cybersecurity dos EUA (2021) e o framework SLSA (*Supply Chain Levels for Software Artifacts*) [SLSA, 2023].

O trabalho está organizado da seguinte forma: a Seção 2 apresenta o embasamento teórico completo, cobrindo assinaturas digitais, o ecossistema Sigstore e cada componente relevante; a Seção 3 descreve o experimento planejado com pseudocódigo e diagrama; e a Seção 4 lista as referências bibliográficas.

2 Desenvolvimento

2.1 Fundamentos de Assinatura Digital

Uma **assinatura digital** é um mecanismo criptográfico que permite a um emissor assinar um documento ou artefato de forma que qualquer receptor possa verificar a autenticidade e a integridade do conteúdo [Schneier, 2015]. O processo baseia-se em criptografia assimétrica, também conhecida como criptografia de chave pública, onde cada participante possui um par

de chaves: uma chave privada, mantida em segredo, e uma chave pública, compartilhada livremente.

O processo de assinatura funciona em duas etapas. Na **etapa de assinatura**, calcula-se o *hash* criptográfico do artefato (tipicamente SHA-256) e aplica-se a chave privada sobre esse hash, gerando a assinatura. Na **etapa de verificação**, o receptor recalcula o hash do artefato recebido, aplica a chave pública sobre a assinatura fornecida, e verifica se os dois valores coincidem. Se coincidirem, tem-se garantia de que: (a) o artefato foi assinado pelo detentor da chave privada correspondente, e (b) o artefato não foi modificado desde a assinatura [Katz & Lindell, 2021].

Algoritmos utilizados pelo Cosign:

Algoritmo	Tipo	Uso no Cosign	Tamanho da chave
ECDSA P-256	Assimétrico	Assinatura padrão	256 bits
Ed25519	Assimétrico	Assinatura alternativa	256 bits
SHA-256	Hash	Digest do artefato	256 bits (saída)
RSA-PSS	Assimétrico	Compatibilidade legada	2048-4096 bits

Tabela 1 – Algoritmos criptográficos utilizados pelo Cosign.

No contexto de assinatura de imagens Docker, o artefato assinado não é a imagem inteira, mas sim seu **digest**, uma representação hash da imagem no formato *sha256:<hash>* gerada pelo registro de containers. Isso garante que a assinatura é vinculada a uma versão exata e imutável da imagem.

2.2 O Problema da Cadeia de Suprimentos de Software

A **cadeia de suprimentos de software** (*software supply chain*) compreende todos os componentes, ferramentas, processos e pessoas envolvidos na produção e entrega de software, desde o código-fonte até o ambiente de produção [NIST SP 800-204D, 2023]. Um ataque a essa cadeia consiste em comprometer qualquer elo dessa sequência para introduzir código malicioso no produto final.

O framework **SLSA (Supply Chain Levels for Software Artifacts)**, desenvolvido pelo Google e mantido pela OpenSSF, categoriza os ataques à cadeia de suprimentos em quatro categorias principais: (A) comprometimento do código-fonte, (B) comprometimento do processo de build, (C) comprometimento dos artefatos gerados, e (D) comprometimento das dependências utilizadas [SLSA, 2023]. O caso XZ Utils enquadra-se nas categorias A e B simultaneamente: o atacante comprometeu o código-fonte e manipulou o processo de build para que os artefatos gerados diferissem do que o código-fonte indicava. A assinatura criptográfica de artefatos endereça principalmente a categoria C, protegendo a integridade dos artefatos após sua geração, durante armazenamento e distribuição.

Níveis de maturidade SLSA:

Nível	Requisito	Garantia oferecida
SLSA 1	Proveniência documentada	Rastreabilidade básica do build
SLSA 2	Build service hospedado	Build não executado localmente
SLSA 3	Build isolado e auditável	Resistência a comprometimento do build
SLSA 4	Revisão de dois mantenedores	Resistência a insider threats

Tabela 2 – Níveis de maturidade do framework SLSA.

A implementação de assinatura criptográfica de artefatos contribui diretamente para atingir o nível SLSA 2, garantindo rastreabilidade e integridade dos artefatos desde o build até o deploy. Contudo, para resistir a ataques do tipo XZ Utils, onde o próprio processo de build é comprometido, seriam necessários os níveis SLSA 3 (build isolado e auditável) e SLSA 4 (revisão obrigatória por dois mantenedores independentes). Este trabalho foca em SLSA 2, que já representa um avanço significativo em relação à ausência total de verificação de integridade.

2.3 Sigstore: Infraestrutura de Confiança Open-Source

Sigstore é um projeto open-source da Linux Foundation que fornece uma infraestrutura pública e gratuita para assinatura, verificação e auditoria de artefatos de software [Sigstore, 2024]. O projeto foi iniciado em 2021 por engenheiros do Google, Red Hat e Purdue University, com o objetivo de tornar a assinatura de software tão simples quanto o uso de HTTPS, eliminando a necessidade de gerenciar chaves criptográficas de longa duração.

O ecossistema Sigstore é composto por três componentes principais que trabalham em conjunto: **Cosign**, responsável pela assinatura e verificação de artefatos; **Fulcio**, uma Autoridade Certificadora que emite certificados de curta duração vinculados a identidades OpenID Connect (OIDC); e **Rekor**, um log de transparência público e imutável que registra todas as operações de assinatura [Newman et al., 2022].

2.4 Cosign: Assinatura de Imagens de Container

Cosign é a ferramenta de linha de comando do ecossistema Sigstore responsável pela assinatura, verificação e gestão de assinaturas de artefatos OCI (*Open Container Initiative*) [Sigstore, 2024]. Ele suporta dois modos principais de operação: assinatura com **chave explícita** (*key-based signing*), onde o usuário gera e gerencia seu próprio par de chaves, e assinatura **sem chave** (*keyless signing*), onde a identidade é verificada via OIDC e o par de chaves é gerado efemeramente para cada assinatura.

No modo keyless, que é o recomendado para pipelines de CI/CD, o Cosign solicita um token OIDC do provedor de identidade da plataforma (por exemplo, GitHub Actions emite tokens OIDC nativamente). Esse token é então apresentado ao Fulcio, que emite um certificado X.509 de curta duração (10 minutos) vinculando a chave pública gerada à identidade do workflow. A assinatura, o certificado e o digest do artefato são registrados no Rekor, tornando a operação auditável publicamente [Newman et al., 2022].

Uma característica importante do Cosign é que as assinaturas são armazenadas no próprio registro de containers (GHCR, Docker Hub, etc.) como **artefatos OCI**, utilizando uma tag derivada do digest da imagem. Isso elimina a necessidade de infraestrutura adicional para armazenar assinaturas e mantém tudo centralizado no registro existente

2.5 Rekor: Log de Transparência Imutável

Rekor é o componente de log de transparência do Sigstore, inspirado no Certificate Transparency (RFC 9162) utilizado para certificados TLS [Laurie et al., 2021]. Ele mantém um registro público, auditável e imutável de todas as operações de assinatura realizadas via Sigstore. A imutabilidade é garantida por uma estrutura de dados chamada **Merkle Tree**, onde cada entrada é encadeada criptograficamente à anterior, tornando qualquer adulteração detectável.

Cada entrada no Rekor contém: o digest do artefato assinado, a assinatura gerada, o certificado do assinante, o timestamp da operação e um identificador único (índice e UUID). Qualquer pessoa pode consultar o Rekor para verificar se uma assinatura é legítima, quando foi criada e por qual identidade, sem depender de uma autoridade centralizada além do próprio log [Newman et al., 2022].

2.6 Fulcio: Autoridade Certificadora sem Chaves de Longa Duração

Fulcio é a Autoridade Certificadora (CA) do ecossistema Sigstore. Seu papel é emitir certificados X.509 de curta duração (10 minutos de validade) vinculando uma chave pública efêmera a uma identidade verificada via OpenID Connect (OIDC). Provedores OIDC suportados incluem GitHub Actions, Google, Microsoft e qualquer provedor compatível com o padrão [Sigstore, 2024].

A grande inovação do Fulcio em relação às CAs tradicionais é eliminar o problema de gerenciamento de chaves de longa duração, que representa um risco de segurança significativo: chaves privadas podem ser roubadas, comprometidas ou mal utilizadas. Como as chaves geradas pelo Cosign no modo keyless têm validade de apenas 10 minutos e são usadas uma única vez, mesmo que uma chave privada seja interceptada, ela é inútil após o término da operação de assinatura [Newman et al., 2022].

3 Experimento Planejado

3.1 Descrição da Implementação

O experimento consistirá na construção de uma cadeia de confiança completa para uma aplicação web simples em Python (Flask), demonstrando o ciclo completo de build, assinatura e verificação de artefatos Docker. O repositório será hospedado no GitHub e utilizará o **GitHub Actions** como plataforma de CI/CD e o **GitHub Container Registry (GHCR)** como registro de imagens.

A escolha de uma aplicação Flask simples como alvo é intencional: o foco do experimento não é a aplicação em si, mas a infraestrutura de confiança em torno dela. A aplicação servirá de veículo para demonstrar como qualquer artefato Docker pode ter sua autenticidade e integridade garantidas criptograficamente.

Componentes do experimento:

Aplicação alvo: API REST simples em Flask com dois endpoints: GET /health e GET /version. Empacotada em imagem Docker com Dockerfile reprodutível.

Pipeline de build e assinatura: Workflow GitHub Actions que executa: build da imagem, push para GHCR, assinatura keyless com Cosign, e registro automático no Rekor.

Pipeline de verificação: Workflow separado que simula o ambiente de deploy: busca a imagem do registry, executa cosign verify com restrições de identidade, e só prossegue se a verificação for bem-sucedida.

Demonstração de adulteração: Cenário onde uma imagem é modificada manualmente após a assinatura, demonstrando que a verificação detecta a adulteração e rejeita o deploy.

Demonstração de imagem não autorizada: Cenário onde uma imagem é assinada por um workflow diferente do autorizado, demonstrando que a política de identidade rejeita a assinatura.

Relatório de auditoria: Script que consulta o Rekor e gera um relatório com o histórico completo de assinaturas: quando, por qual workflow, e com qual digest.

3.2 Pseudocódigo e Diagrama do Pipeline

Diagrama de componentes do experimento:

Componente	Tecnologia	Função	Resultado gerado
Aplicação alvo	Flask + Docker	Artefato a ser protegido	Imagem OCI
CI/CD	GitHub Actions	Orquestração dos workflows	Logs de execução
Registry	GHCR	Armazenamento de imagem e assinatura	Imagem + .sig
Assinatura	Cosign v2	Geração da assinatura ECDSA	Entrada no Rekor
Identidade	Fulcio CA	Certificado efêmero (10min)	Cert. X.509
Auditoria	Rekor	Log imutável público	Entrada com UUID
Verificação	Cosign verify	Validação antes do deploy	OK ou ERRO
Relatório	Script Python	Histórico de assinaturas	JSON + HTML

Tabela 3 – Componentes do experimento, tecnologias utilizadas e resultados esperados.

Os cenários de demonstração planejados são: (1) **fluxo feliz**, imagem construída pelo pipeline oficial, assinada e verificada com sucesso antes do deploy; (2) **imagem adulterada**, imagem modificada após a assinatura, verificação detecta divergência de digest e rejeita; (3) **assinante não autorizado**, imagem assinada por um workflow diferente, política de identidade rejeita a assinatura; e (4) **relatório de auditoria**, consulta ao Rekor mostrando o histórico completo e rastreável de todos os builds e assinaturas do projeto.

4 Referências Bibliográficas

[CISA, 2024] CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY. *Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094*. CISA Advisory, abr. 2024. Disponível em: <https://www.cisa.gov/news-events/alerts/2024/03/29/reported-supply-chain-compromise-affecting-xz-utils>.

[Katz & Lindell, 2021] KATZ, J.; LINDELL, Y. *Introduction to Modern Cryptography*. 3. ed. CRC Press, 2021.

[Laurie et al., 2021] LAURIE, B. et al. *RFC 9162: Certificate Transparency Version 2.0*. IETF, 2021. Disponível em: <https://www.rfc-editor.org/rfc/rfc9162>. Acesso em: 28 jun. 2025.

[Newman et al., 2022] NEWMAN, Z. et al. Sigstore: Software signing for everybody. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, 2022. p. 2353-2367.

[NIST SP 800-204D, 2024] CHANDRAMOULI, R.; KAUTZ, F.; TORRES-ARIAS, S. *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. NIST Special Publication 800-204D. National Institute of Standards and Technology, Gaithersburg, MD, fev. 2024. DOI: 10.6028/NIST.SP.800-204D. Disponível em: <https://csrc.nist.gov/pubs/sp/800/204/d/final>.

[Schneier, 2015] SCHNEIER, B. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. 20th Anniversary Ed. Wiley, 2015.

[Sigstore, 2024] SIGSTORE PROJECT. *Sigstore Documentation*. Linux Foundation, 2024. Disponível em: <https://docs.sigstore.dev>.

[SLSA, 2023] OPEN SOURCE SECURITY FOUNDATION. *SLSA: Supply Chain Levels for Software Artifacts – Framework Specification v1.0*. OpenSSF, 2023. Disponível em: <https://slsa.dev/spec/v1.0>.

